Stela System Architecture and Governance Analysis: A Comprehensive Audit of Pointer-First Agent Constitution and Operational Rigor

1. Introduction: The Paradigm of Constitutional AI Governance

The evolution of artificial intelligence in software engineering has necessitated a fundamental shift from unstructured, prompt-based interactions to rigorously governed, constitution-first architectures. The Stela platform represents a sophisticated implementation of this paradigm, characterized by its "Pointer-First" Agent Constitution and a strict separation of concerns defined by its "Filing Doctrine." This report provides an exhaustive, forensic audit of the Stela repository, specifically focusing on the recently installed Agent Sub-system, its alignment with the Policy of Truth (PoT), and its operational rigor compared to the existing Skills Sub-system.

The analysis draws upon a comprehensive "repo dump" containing the core governance files—PoT.md, SoP.md, TASK.md—operational binaries within ops/bin/, and the newly refactored agent library residing in docs/library/agents/.[1] The central thesis of this examination is that Stela implements a governance model where agents do not possess intrinsic, hallucination-prone instructions. Instead, they function as lightweight pointers that reference immutable governance documents and Just-In-Time (JIT) skills, ensuring that the logic of the AI remains subservient to the authority of the human Operator.[1]

This audit is structured to address four critical inquiries regarding the system's integrity: the comparative rigor of the agent versus skill subsystems, the alignment of the new agents with the Policy of Truth, the interoperability of agents with tasks and skills, and a security review for potential malicious code injection. By deconstructing the repository's "Physical Laws" and "Behavioral Logic Standards," this report establishes a verifiable baseline for the system's readiness for production deployment.[1]

2. The Constitutional Framework: Deconstructing the Policy of Truth

The foundation of the Stela architecture is the **Policy of Truth (PoT)**. This document is not merely a set of guidelines but a "constitutional" file that defines the physical laws of the repository.[1] Understanding the PoT is prerequisite to auditing the Agent Sub-system, as the

agents are designed explicitly to function as executive arms of this constitution. The PoT establishes a non-negotiable hierarchy of authority and a strict classification of digital artifacts.

2.1 The Doctrine of Filing and Jurisdiction

The PoT establishes a "Filing Doctrine" that separates repository content into three distinct jurisdictions: **Run**, **Explain**, and **Work**. This tripartite separation is critical for AI governance, as it prevents agents—who primarily operate in the projects/ directory—from rewriting the fundamental rules of the operational environment.[1]

Jurisdiction	Directory	Definition	Governance Rule
Run	ops/	Binaries, manifests, automation.	The "Operating System" of the repo. Only authoritative logic lives here.
Explain	docs/	Manuals, rationale, registries.	Passive documentation. Cannot execute logic.
Work	projects/	Payload code.	The product. Subject to PoT but distinct from the Platform.
Trash	storage/	Local artifacts.	Never canon. Used for handoffs and dumps.

This separation creates a "security air gap" between the logic that runs the platform and the logic that runs the product. The audit of the new Agent Sub-system confirms it respects this doctrine: the agent definitions reside in docs/library/agents/ (Explain), while the lifecycle logic resides in ops/lib/scripts/agent.sh (Run).[1] This structural compliance ensures that an agent cannot accidentally modify its own lifecycle logic while attempting to update its documentation, a common failure mode in self-referential AI systems.

2.2 The Staffing Protocol: The Tripartite Model

Stela enforces a strict "Staffing Protocol" that defines the relationship between human authority and AI logic.[1] This protocol is hard-coded into the operational directives of the agents and serves as the primary defense against unauthorized autonomy.

1. **Operator (Human):** The Operator is the sole sovereign of the repository. They own final decisions, approvals, and secrets. Crucially, the PoT states the Operator "Performs all commits, pushes, and merges".[1] This is a security firewall; no AI agent is permitted to write directly to the version control history. This constraint is reinforced by the .github/copilot-instructions.md file, which explicitly forbids direct pushes to main.[1]
2. **Integrator (Lead AI):** The Integrator functions as the "Guardian of Governance." Its role is to maintain structural integrity and detect "Drift" (divergence from the PoT). The Integrator generates "Dispatch Packets" (DPs) but does not execute the code itself.[1] This role creates a layer of oversight within the AI workforce itself, ensuring that tasks are structurally sound before execution begins.
3. **Contractor (Guest AI):** The Contractor is the execution arm. It drafts logic and implementation details within a defined scope. It is explicitly forbidden from creating or switching branches.[1] This limitation ensures that the Contractor cannot "escape" the sandbox defined by the Integrator and Operator.

The "Pointer-First" constitution relies on this hierarchy. Agents defined in docs/library/agents/ (like R-AGENT-01 or R-AGENT-04) function as specialized Contractors or Integrators. They do not have autonomy; they have "contracts" defined by TASK.md, ensuring that every CPU cycle spent by an agent is authorized by a specific Dispatch Packet.[1]

2.3 Behavioral Logic Standards and Anti-Drift Governance

The PoT imposes strict "Behavioral Logic Standards" on all agents.[1] These standards are designed to minimize token ambiguity and maximize auditability, transforming natural language processing into a more deterministic execution flow.

- **Linguistic Precision:** Agents are prohibited from using contractions (e.g., "don't," "can't"). This forces a formal, rhythmic cadence that makes technical auditing easier and reduces the likelihood of an LLM misunderstanding a negated command.[1] The phrase "do not" is unambiguous; "don't" can be tokenized in ways that obscure the negation in complex prompts.
- **Quantitative Reporting:** Deviations must be reported numerically (e.g., "execution path deviated by 14.7%"). This prevents vague, "vibes-based" reporting and forces the agent to measure its performance against a concrete standard.[1]
- **Absolute Literalism:** Agents must interpret instructions with total fidelity. Ambiguity is a stop condition, not an invitation to improvise. This standard compels the agent to halt execution and request clarification if the prompt allows for multiple interpretations, preventing "hallucinated compliance".[1]

This standard is the primary mechanism for "Anti-Drift Governance." If an agent begins to

speak casually or interpret rules loosely, it is immediately detectable as a system failure. The system treats "Drift" not as a warning but as a critical error requiring immediate correction.[1]

3. The Pointer-First Agent Constitution: Deep Analysis

The core innovation of the new installation is the "Pointer-First" architecture found in docs/library/agents/. In traditional AI agent frameworks, the "system prompt" is often a massive, monolithic block of text containing all rules, context, and capabilities. This approach is prone to "context washing," where rules at the beginning of the prompt are forgotten as the context window fills. Stela employs a different strategy: the agent definition is a skeletal structure that *points* to the source of truth rather than containing it.

3.1 Anatomy of a Stela Agent

An audit of docs/library/agents/R-AGENT-01.md (Architect) reveals the rigorous structure mandated by the new system [1]:

Agent: architect

Provenance

- **Captured:** 2026-02-06 21:21:15 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
  ...

Role

Software architecture specialist...

Pointers

- Constitution: PoT.md
- Governance/Jurisdiction: docs/GOVERNANCE.md
- Output contract: TASK.md
- Authorized toolchain: ops/bin/open, ops/bin/dump...
- JIT skills:
  - docs/library/skills/S-LEARN-02.md
  - docs/library/skills/S-LEARN-03.md
  - docs/library/skills/S-LEARN-06.md

Analysis of the Pointer-First Approach:

1. **Immutable Law via Reference:** The agent does not "know" the laws; it points to PoT.md.

This means if the Constitution changes, the agent automatically aligns with the new law without needing to be re-prompted or re-coded. This solves the "Stale Governance" problem inherent in static system prompts.[1]

2. **Just-In-Time (JIT) Skills:** The agent does not "know" how to perform a specific task (like S-LEARN-06 Advanced Git Forensics). It points to the skill file. This allows for modular skill upgrades. If the forensics technique improves, the agent uses the new method immediately, ensuring that the "best known method" is always applied.[1]
3. **Toolchain Authorization:** The agent lists its "Authorized toolchain." This acts as an allowlist, preventing the agent from hallucinating access to tools it does not have (e.g., npm or docker commands that aren't wrapped in ops/ binaries). This restricts the agent's surface area for potential damage.[1]

3.2 Deep Dive: Agent Registry Audit

The audit examined the six registered agents to verify their alignment with the Staffing Protocol. The registry at docs/ops/registry/AGENTS.md acts as the authoritative roll call for the AI workforce.[1]

Agent ID	Name	Role (Derived from)	Pointer Analysis
R-AGENT-01	Architect	System design, scalability, technical decision-making.	Points to S-LEARN-03 (Zenith Reference) and S-LEARN-06 (Git Forensics). This aligns with the "Integrator" persona of high-level planning.
R-AGENT-02	Code-Reviewer	Quality, security, maintainability review.	Points to S-LEARN-04 (Coding Standards) and S-LEARN-01 (Verification Loop). Strictly a "Contractor" role focused on payload quality.

R-AGENT-03	Doc-Updater	Documentation and codemap specialist.	Points to S-LEARN-08 (Prune SoP). This agent manages the "Explain" layer (docs/), ensuring documents match the code.
R-AGENT-04	Integrator	Planning and integration for complex changes.	Points to S-LEARN-07 (Harvest Skills). This agent is the primary driver of the "Integrate" command and DP generation.
R-AGENT-05	Refactor-Cleaner	Dead code cleanup and consolidation.	Points to S-LEARN-05 (Security Protocols). Uses analysis tools like knip and depcheck (referenced in B-TASK-05).
R-AGENT-06	Security-Reviewer	Vulnerability detection and remediation.	Points to S-LEARN-05 (Security Protocols). Focused on OWASP and secret detection.

Insight: The segregation of duties is absolute. The **Architect** does not write code; the **Refactor-Cleaner** does not plan architecture. This prevents "Role Drift," where an AI attempts to solve problems outside its context window's competency. The explicit provenance tracking (capturing the exact DP-ID and Git Hash where the agent was defined) ensures that if an agent behaves erratically, its origin point can be audited and corrected.[1]

3.3 Context Hygiene and Hazard Avoidance

A critical finding in the PoT is the concept of a **Context Hazard**.[1] The rule states that ops/lib/manifests/CONTEXT.md must *exclude* docs/library/agents, docs/library/tasks, and docs/library/skills.

- **The Logic:** If an agent's context window contains the definitions of all other agents, it risks "Identity Confusion" or recursive logic loops (e.g., an agent calling itself). Furthermore, loading the entire library consumes valuable context tokens that should be reserved for the task payload.
- **The Implementation:** Stela uses tools/lint/context.sh and ops/bin/llms to enforce this. The llms binary specifically includes a "Hazard Exclusion" routine that rejects any bundle path resolving to the library directories.[1]

This proves the system is designed for **stateless rehydration**. An agent is hydrated with only the *pointers* it needs for the specific TASK.md at hand, rather than the entire universe of potential agents. This improves reliability by reducing noise in the inference window.

4. Operational Rigor: The Scripts (agent.sh vs skill.sh)

The rigor of the Stela system is enforced by the "Run" layer in ops/. We audited the two primary lifecycle scripts: ops/lib/scripts/agent.sh and ops/lib/scripts/skill.sh.[1] The user query specifically asks if the Agent system meets the same rigor as the Skills system.

4.1 skill.sh: The Provenance Engine

The skill.sh script is the enforcement mechanism for the "Contractor closeout skill harvesting" rule.[1] It is not merely a file creation script; it is a forensic tool.

- **Harvesting Logic:** The script uses heuristics.sh to detect "Hot Zones" (files with high change volume) and "High Churn" (files changed frequently). It uses this data to auto-generate a Provenance block for every new skill.
  - *Code Evidence:* generate_provenance_block in heuristics.sh captures timestamp, DP-ID, Branch, HEAD, Objective, and Diff Stat.[1] This ensures that every piece of knowledge in the system has a traceable origin.
- **Semantic Collision Check:** The script includes a check_semantic_collision function that scans existing skills for keyword overlaps. This prevents the repository from becoming cluttered with duplicate skills (e.g., "Fix Bug" vs "Bug Fix"), ensuring a clean namespace for JIT loading.[1]
- **Promotion Gate:** The promote function moves a draft from storage/handoff/ to docs/library/skills/. Crucially, it creates an immutable "Promotion Packet" in docs/library/SKILLS.md (the ledger) and updates docs/library/INDEX.md.[1] This triple-bookkeeping ensures that the Skill file, the Index, and the Ledger are always synchronized.

Rigor Assessment: The skill.sh script meets the highest standard of rigor. It removes human

(or AI) laziness from the equation. An agent cannot simply "write a doc"; it must "harvest a skill," which forces the system to record *where* that skill came from (the DP and Git state).

4.2 agent.sh: The Staffing Automaton

The agent.sh script manages the lifecycle of the AI agents themselves.[1] It mirrors the rigor of skill.sh but adds specific checks relevant to agent governance.

- **Guardrails:** The cmd_check_guardrail function performs two vital checks:
  1. **Scope Boundary:** It verifies that every agent file contains a ## Scope Boundary section. This ensures no agent exists without defined limits.
  2. **Context Hazard Check:** It greps for docs/library/agents inside the agent definitions. This prevents an agent from referencing the agent library itself, enforcing the "Context Hygiene" rule.[1]
- **Harvesting Agents:** Similar to skills, agents are "harvested" from a DP. If a specific prompting strategy worked well in DP-OPS-0029, it is harvested into R-AGENT-04. The script auto-generates the Provenance block, linking the agent's existence to a specific successful work cycle.[1]
- **Linguistic Linter:** The script includes pot_duplication_linter, which checks for forbidden phrases like "No contractions" or "Policy of Truth."
  - *Insight:* This is a meta-check. It ensures agents *point* to the PoT (PoT.md) rather than *repeating* the PoT. This reduces token bloat and ensures that if the PoT changes, the agents don't need to be updated. It forces the "Pointer-First" constitution into existence.[1]

4.3 Comparative Analysis: Agent vs. Skill Systems

Feature	skill.sh	agent.sh
Provenance	Mandatory. Captures Hot Zones.	Mandatory. Captures Source DP.
Validation	Semantic Collision Check.	Context Hazard Check (Guardrail).
Linter	Checks for placeholders (TODO).	Checks for PoT Duplication.
Ledger	Updates SKILLS.md.	Updates AGENTS.md (Registry).

SoP Impact	Updates SoP.md on promotion.	Updates SoP.md on promotion.

Verdict on Rigor: Both scripts exhibit identical high rigor. They are designed to prevent "Drift" by ensuring that no artifact enters the docs/ library without a traceable history (provenance) and a clean bill of health (linting). The Agent system is *as secure and rigorous* as the Skills system, and in some ways more so due to the pot_duplication_linter which specifically enforces the "Pointer-First" philosophy.

5. Interoperability and Combinatorial Logic

The Stela system is designed for modular interoperability. Agents (R), Skills (S), and Tasks (B) combine to form execution units. This combinatorial logic allows for a highly flexible yet governed workflow.

5.1 The Integration Workflow (B-TASK-03)

The B-TASK-03 file (Integrate Command) defines how these components combine to execute complex workflows.[1]

- **Workflow:** integrator -> code-reviewer -> security-reviewer.
- **Logic Flow:**
  1. The **Integrator** (R-AGENT-04) activates. It loads S-LEARN-07 (Harvest Skills) to plan the work and ensure that any new knowledge generated is captured.
  2. It generates a Handoff Document, summarizing the plan.
  3. The **Code-Reviewer** (R-AGENT-02) activates, loading S-LEARN-04 (Coding Standards). It uses this skill to audit the code against the defined standards, ensuring no "any" types in TypeScript or unformatted Python code.
  4. The **Security-Reviewer** (R-AGENT-06) activates, loading S-LEARN-05 (Security Protocols). It scans the code for secrets, RLS violations, or CORS misconfigurations.
- **Combinatorial Power:** By decoupling the *role* (Agent) from the *knowledge* (Skill), Stela allows for dynamic recomposition. If a new security protocol is defined in S-LEARN-05, the **Security-Reviewer** automatically applies it without needing a prompt update or a new agent definition.[1]

5.2 The Plan Command (B-TASK-04)

The B-TASK-04 file (Plan Command) demonstrates the "Stop Condition" logic and the separation of planning from execution.[1]

- **Procedure:** The Integrator analyzes the request, breaks it down into phases, identifies dependencies, and assesses risks.
- **Constraint:** "The integrator agent will **NOT** write any code until you explicitly confirm the plan." This is a hard stop condition.

- **Alignment with PoT:** This enforces the "Operator Authority" rule. The AI drafts the plan, but the Human must approve it before any work/ branch is modified. This interaction model prevents the AI from "running away" with a task based on a misunderstanding of the objective.[1]

5.3 Task Discovery and Execution

The TASK.md file serves as the "active work surface" where these combinations are instantiated. The "Required Context Load" section of TASK.md forces the worker to explicitly confirm the loading of PoT.md, SoP.md, and specific skills. This manual confirmation step (echoing "Loaded: PoT...") acts as a "freshness gate," ensuring that the agent is not operating on stale memory or hallucinated context.[1]

6. Security Audit and Malicious Code Check

A primary requirement of this report is to verify that the new agent system does not introduce malicious logic, specifically regarding git operations and repo-gate bypasses. The audit focused on identifying any mechanism that would allow an AI agent to usurp the Operator's authority.

6.1 Git Authority Audit

We audited ops/lib/scripts/agent.sh and ops/lib/scripts/skill.sh for unauthorized git commands.

- **Findings:**
  - The scripts use git rev-parse, git status, git diff, git log, git blame, and git ls-files. These are **read-only** forensic commands used to generate provenance and check state.[1]
  - There are **NO** instances of git push, git commit, git merge, or git fetch in the scripts.
  - The scripts modify local files (SoP.md, INDEX.md, SKILLS.md) on the disk, but they rely on the **Operator** to manually stage (git add), commit, and push these changes.
- **Conclusion:** The system strictly adheres to the Staffing Protocol. The AI (via the script) prepares the state (staged files), but the Human must pull the trigger (commit/push). There is no "auto-commit" logic that could insert malicious code into the history without human review.

6.2 Repo-Gate Bypass Audit

We checked for logic that might disable or skip verification steps or CI/CD pipelines.

- **Findings:**
  - The agent.sh script *adds* a guardrail (cmd_check_guardrail) rather than removing one. It actively checks for context hazards before allowing an agent to be promoted.
  - The skill.sh script includes a verification step in the "Definition of Done" block of the Promotion Packet: bash tools/verify.sh.[1] It explicitly requires the user to run

verifications.
    - The repo_gates.yml workflow enforces tools/lint/truth.sh and tools/lint/library.sh. The scripts in ops/ are designed to *pass* these linters (e.g., by ensuring INDEX.md is updated), not to disable them. For example, agent.sh automatically adds the new agent to INDEX.md to prevent the Library Guard from failing.[1]
- **Malicious Code Verdict: Negative.** The system is designed to *increase* friction for non-compliant code, not to bypass it. The scripts act as compliance assistants, automating the bureaucracy required to pass the gates.

6.3 Anti-Drift Verification

The tools/verify.sh script was audited to ensure it captures the new agent files.[1]

- It checks for the existence of docs, ops, tools.
- It checks for storage hygiene, ensuring no unauthorized files clutter the repo.
- **Library Guard (tools/lint/library.sh):** This script verifies that every file in docs/library/ is registered in INDEX.md. This prevents "Ghost Agents" (agents that exist on disk but are not governed/indexed). The agent.sh script automatically updates INDEX.md upon promotion, ensuring compliance. This creates a closed loop where unauthorized files are detected by CI/CD.[1]

7. Alignment with the Policy of Truth (PoT)

The final verification is whether the Agent Sub-system aligns with the Constitution. We mapped the specific PoT mandates to the implementation details of the Agent Sub-system.

PoT Mandate	Agent System Implementation	Verdict
Filing Doctrine (1.1)	Agents reside in docs/ (Explain), Scripts in ops/ (Run).	**COMPLIANT**
SSOT Pointer (1.2)	Agents point to PoT.md and TASK.md instead of duplicating text.	**COMPLIANT**
No Contractions (4.2)	Scripts and agent definitions enforce "Linguistic Precision" (no don't, can't).	**COMPLIANT**

Context Hygiene (4.2)	agent.sh scans for Context Hazards (docs/library/agents references).	**COMPLIANT**
Provenance (4.2)	skill.sh and agent.sh capture DP-ID and Git Hash in provenance blocks.	**COMPLIANT**
Equilibrium (4.2)	Scripts update SoP.md on promotion, maintaining history.	**COMPLIANT**
Operator Authority (4.1)	No git push capability in scripts; AI is read-only on git history.	**COMPLIANT**

Drift Note: The docs/ops/registry/AGENTS.md file is located in docs/ops/, which is consistent with the "Explain" jurisdiction (registries explain what exists). However, PoT.md says ops/ is for "Run." This is a minor nuance: registries are static documentation *of* the operational layer, so placing them in docs/ops/ is the correct interpretation of the Filing Doctrine, maintaining the separation between the executable ops/ and the descriptive docs/.[1]

8. Gap Analysis and Recommendations

While the system is highly rigorous, the audit identified areas for potential enhancement to further strengthen the governance model.

8.1 Gap: Registry Redundancy

The file docs/ops/registry/AGENTS.md exists in docs/ops/. However, docs/library/INDEX.md *also* acts as a registry for all library files.

- **Risk:** Double-maintenance. If agent.sh updates one but not the other, the registries drift.
- **Mitigation:** The agent.sh script currently updates *both* AGENTS.md and INDEX.md, mitigating the immediate risk.
- **Recommendation:** Deprecate AGENTS.md and use INDEX.md as the single SSOT for agent registration, or define AGENTS.md as the "Deep Registry" (containing metadata like model version and tool access) and INDEX.md as the "Navigation Registry" (pointers only).

8.2 Gap: Skill Discovery

The skill.sh script checks for semantic collisions, but there is no "Semantic Search" for skills. As the library grows to 100+ skills, agents might fail to find the correct JIT skill simply because they don't know the file name.

- **Recommendation:** Create an ops/bin/skills binary (similar to ops/bin/llms) that generates a skills-map.txt for agents to "browse" the available capabilities without loading the full content. This would align with the "Pointer-First" philosophy by giving agents a map of pointers to capabilities.

8.3 Gap: Automated Agent Testing

Currently, the verification of an agent is static (linting). There is no "functional test" to ensure an agent performs its role correctly.

- **Recommendation:** Implement a tools/test/agent_behavior.sh script that runs a mock task against an agent and verifies its output format (e.g., checking if it actually used quantitative reporting).

9. Conclusion

The audit confirms that the Stela Agent Sub-system is a **high-rigor, secure, and constitutionally aligned** architecture. It successfully implements the "Pointer-First" design pattern, effectively solving the problems of context drift and hallucinated governance.

The system places "Governance as Code" at its core. By treating the Policy of Truth not as text but as a dependency that must be linked, linted, and verified, Stela achieves a level of operational reliability that is rare in AI-assisted development.

Final Status Verdicts:

1. **DOES THE SYSTEM MEET THE SAME RIGOR AS SKILLS? Yes.** The agent.sh script mirrors the provenance and validation logic of skill.sh, introducing specific guardrails for Context Hazards that make it equally rigorous.
2. **IS THE SYSTEM IN ALIGNMENT WITH THE POLICY OF TRUTH? Yes.** The architecture enforces the Filing Doctrine, Staffing Protocol, and Behavioral Logic Standards without deviation.
3. **WILL THEY WORK IN DIFFERENT COMBINATIONS? Yes.** The decoupling of Agents (Role) from Skills (Capability) and Tasks (Objective) allows for dynamic, modular execution flows, as evidenced by B-TASK-03.
4. **DID THE MODEL INSERT MALICIOUS CODE? No.** A forensic audit of the scripts reveals no unauthorized git operations or mechanism to bypass repo-gates. The system maintains the Operator's exclusive authority over the repository state.

The system is validated for production use under the supervision of a Human Operator. The implementation represents a significant advancement in the field of governed AI

architectures.

Detailed Analysis of Repository Artifacts

1. Introduction to the Stela Ecosystem

The Stela repository operates on a unique "Constitution-First" governance model, codified in the **Policy of Truth (PoT)**. Unlike traditional repositories where governance is implicit (tribal knowledge) or scattered (wikis, READMEs), Stela treats governance as executable code. The PoT.md file is not just a guide; it is the "Physical Law" of the environment.[1]

This report audits the newly installed **Agent Sub-system**, which introduces "Pointer-First" AI agents into this ecosystem. The goal is to determine if these agents strengthen the governance model or introduce "Drift" (entropy). The analysis is based on the provided dump of the repository state at HEAD ab61ce66b.[1]

1.1 The "Pointer-First" Concept

In the Stela context, "Pointer-First" means that an AI agent's definition file (R-AGENT-xx.md) contains almost no intrinsic knowledge. Instead, it contains **pointers** to:

1. **The Law:** PoT.md (Constitution).
2. **The Contract:** TASK.md (Current objective).
3. **The Capability:** S-LEARN-xx.md (JIT Skills).
4. **The Toolchain:** ops/bin/ (Executables).

This architecture minimizes "hallucination" by ensuring the agent's context is always anchored to the repository's Single Source of Truth (SSOT). An agent cannot "forget" the rules because it does not "remember" them; it "reads" them fresh every session.[1]

2. Governance Architecture (The PoT)

To audit the agents, we must first define the laws they must follow. The PoT.md file outlines the "Physical Laws" of the repository.

2.1 The Filing Doctrine

The PoT establishes a rigid directory structure [1]:

```
/
├── ops/       # RUN: The executable kernel (binaries, scripts).
├── docs/      # EXPLAIN: The manual and registry.
├── projects/  # WORK: The payload.
└── storage/   # TRASH: Local, ephemeral artifacts.
```

Audit Finding: The new agents are located in docs/library/agents/. This aligns with the doctrine because an agent definition is a "Manual" for the AI role—it explains *who* the agent is. The *logic* for managing agents is in ops/lib/scripts/agent.sh, which aligns with the "Run" jurisdiction. The separation is clean and compliant.

2.2 The Axioms of Governance

The PoT lists several axioms that agents must respect [1]:

1. **Context Hazard:** Agents must never load the entire agent library into their context. This prevents "recursive identity" issues. The audit of ops/bin/llms confirms that it strips docs/library/agents from the bundle, enforcing this axiom via code.
2. **Reuse-First:** Agents must use existing templates. The agent.sh script enforces this by using a standardized template for all new agent drafts.
3. **Equilibrium:** A task ends only when SoP.md is updated. The agent.sh script automatically appends an entry to SoP.md when an agent is promoted, ensuring equilibrium is maintained programmatically.

3. The Agent Sub-System (Deep Dive)

The docs/library/agents/ directory contains the persona definitions. We analyzed the "Provenance" and "Pointers" of these agents to verify their integrity.

3.1 R-AGENT-01: The Architect

- **Role:** High-level system design.
- **Pointer Analysis:**
  - Points to S-LEARN-03 (Zenith Reference Spec). This ensures the Architect uses the approved API patterns ({ ok: boolean, data: T }) defined in Zenith, rather than inventing new ones.[1]
  - Points to S-LEARN-06 (Git Forensics). This allows the Architect to look at "Hot Zones" (areas of high churn) to inform refactoring decisions.
- **Compliance:** The Architect is strictly a "Explain/Plan" agent. It does not have write access to projects/ code in its primary definition, aligning with the "Integrator" role.[1]

3.2 R-AGENT-04: The Integrator

- **Role:** The "Lead AI." Governance and drift detection.

- **Pointer Analysis:**
  - Points to S-LEARN-07 (Harvest Skills). This is the critical recursive loop. The Integrator is the agent responsible for identifying new skills and calling the harvest script.
  - Points to S-LEARN-08 (Prune SoP). This ensures the Integrator handles the hygiene of the history ledger.[1]
- **Compliance:** The Integrator is the only agent authorized to propose branch names (via B-TASK-04), which aligns with the Staffing Protocol.[1]

3.3 R-AGENT-06: The Security Reviewer

- **Role:** Vulnerability detection.
- **Pointer Analysis:**
  - Points to S-LEARN-05 (Security Protocols). This file contains the hard rules for RLS (Row Level Security) and CORS.
  - **Anti-Drift:** If the S-LEARN-05 file is updated to include a new vulnerability check, R-AGENT-06 immediately adopts it. This is the power of the Pointer-First model—instant, global updates to agent behavior without retraining.[1]

4. Operational Scripts: agent.sh and skill.sh

The ops/ directory contains the "Run" logic. We compared agent.sh and skill.sh for consistency and rigor.

4.1 Provenance Generation (heuristics.sh)

Both scripts rely on ops/lib/scripts/heuristics.sh (shared logic).[1]

- **Hot Zone Detection:** git diff --numstat identifies which files are changing the most.
- **High Churn Detection:** git log identifies files with frequent commits.
- # Provenance Block: Provenance
  - Captured: 2026-02-06
  - DP-ID: DP-OPS-0029
  - Friction Context: Hot Zone: Universal / Payload Verification
    This block is auto-injected into every new Skill and Agent. It creates an unbroken chain of custody for knowledge. We know *why* a skill exists (it solved a specific friction in a specific DP).

4.2 Guardrails and Linters

agent.sh Guardrails: [1]

1. **Scope Boundary Check:** Ensures the agent definition explicitly limits its own power (##

Scope Boundary).
2. **PoT Duplication Linter:** Scans for phrases like "Policy of Truth" in the body text. If found, it fails. The agent must *link* to the PoT, not *copy* it. This enforces the SSOT axiom.

skill.sh Guardrails: [1]

1. **Placeholder Check:** Fails if TODO or ENTER_ is found. This enforces "Linguistic Precision" (the work must be finished).
2. **Semantic Collision:** Checks if a skill with a similar name already exists. This prevents "Library Bloat" and maintains a clean namespace.
3. **Context Hazard:** Checks ops/lib/manifests/CONTEXT.md to ensure no skills are globally loaded.

Comparison:

Both scripts are rigorous. skill.sh focuses on **uniqueness and utility** (preventing duplicates), while agent.sh focuses on **governance and jurisdiction** (preventing drift). This distinction correctly reflects the different nature of Agents (governance entities) vs. Skills (utility entities).

5. Combinatorial Logic (The "Working Memory" Model)

The Stela architecture solves the "Context Window Limit" problem through combinatorial logic.

5.1 The Problem

An LLM cannot load the entire repository (docs/, ops/, projects/) into its context window without losing focus or hallucinating.

5.2 The Stela Solution

1. **The Base Context:** llms-small.txt loads the Constitution (PoT, SoP, TASK). This is the "BIOS" of the agent.[1]
2. **The Agent Context:** The R-AGENT-xx file loads the specific "Persona" and "Toolchain."
3. **The Skill Context:** The Agent "JIT loads" only the specific S-LEARN-xx files needed for the task.

Example: B-TASK-03 (Integrate) [1]

- **Step 1:** Load PoT + R-AGENT-04 (Integrator). *Context: Governance.*
- **Step 2:** Integrator decides S-LEARN-05 (Security) is needed.
- **Step 3:** Load R-AGENT-06 (Security Reviewer) + S-LEARN-05. *Context: Security.*
- **Step 4:** Execution.

This "Paging Mechanism" ensures the agent always has high-resolution context for the

immediate sub-task, while maintaining low-resolution awareness of the global constitution.

6. Forensic Security Audit

We examined the dump for any evidence of "Malicious Logic" or "Staffing Protocol Violations."

6.1 The "No Push" Guarantee

The most critical rule in Stela is that **AI never pushes**.

- **Audit:** Grepping ops/lib/scripts/*.sh for git push.
- **Result:** 0 matches.
- **Audit:** Grepping for git commit.
- **Result:** 0 matches.
- **Mechanism:** The scripts rely on the **Operator** to perform the final git add. && git commit. The scripts only manipulate the filesystem (staging area). This guarantees a "Human-in-the-Loop" for every state change.[1]

6.2 The "Repo-Gate" Firewall

- **Threat:** An agent might try to disable the linters in .github/workflows/.
- **Defense:**
  1. **Workflows are Canon:** repo_gates.yml is protected by sop_policing.yml. If an agent touches it, task_policing.yml fires, requiring a TASK update, which triggers a human review.[1]
  2. **Linter Supremacy:** The tools/verify.sh script is run by CI/CD, not the agent. The agent cannot "skip" the CI/CD run.
  3. **Stop Condition:** The TASK.md contract explicitly states: "STOP if told to create or switch branches." This prevents the agent from creating a rogue branch to bypass gates.[1]

7. Comparative Analysis Table: Agent vs. Skill Subsystems

The following table summarizes the findings of the comparative audit between the Agent and Skill subsystems, confirming that the new Agent system matches the rigor of the existing Skill system.

Dimension	**Skill Sub-System (skill.sh)**	**Agent Sub-System (agent.sh)**	**Alignment Verification**
Governance	docs/library/skills/	docs/library/agents	**Aligned** (Both

Jurisdiction	(Explain)	/ (Explain)	respect Filing Doctrine)
Automation Logic	ops/lib/scripts/skill.sh (Run)	ops/lib/scripts/agent.sh (Run)	**Aligned** (Both reside in ops/)
Provenance Data	Timestamp, DP-ID, Hot Zone, Churn	Timestamp, DP-ID, Branch, HEAD	**Aligned** (High-fidelity tracking)
Naming Convention	S-LEARN-XX	R-AGENT-XX	**Aligned** (Systematic ID generation)
Collision Detection	Semantic Keyword Overlap	N/A (Agents are unique singletons)	**Divergent** (Appropriate to domain)
Safety Checks	Context Hazard Scan (Global)	Context Hazard Scan (Self-Ref)	**Aligned** (Both prevent loops)
Linguistic Enforcement	Checks for TODO/ENTER_	Checks for PoT duplication	**Aligned** (Enforces rigor)
Ledger Update	SKILLS.md and INDEX.md	AGENTS.md and INDEX.md	**Aligned** (Double-bookkeeping)
History Equilibrium	Updates SoP.md	Updates SoP.md	**Aligned** (Maintains state history)

8. Final Recommendation

The audit concludes that the Stela Agent Sub-system is robust, secure, and constitutionally aligned. It meets the "Policy of Truth" standards and integrates seamlessly with the existing Skill and Task architectures. The operational scripts prevent malicious behavior through strict read-only git policies and automated guardrails.

Next Steps for the Operator:

1. **Activate Agents:** The agents R-AGENT-01 through R-AGENT-06 are ready for deployment in TASK.md assignments.
2. **Monitor Drift:** Use tools/verify.sh regularly to ensure the new agents do not introduce file-system drift.
3. **Refine Registry:** Consolidate the AGENTS.md and INDEX.md logic in a future DP to reduce redundancy.

The system is green for operation.

Works cited

1. dump-full-main-ab61ce66b.txt